

PostgreSQL — конспект для собеседования

Сводка тем, которые разбирались в подготовке: SQL и операции, JOIN/EXISTS, индексы, B-tree и составные индексы, EXPLAIN/планировщик/статистика/ANALYZE, транзакции и изоляция, MVCC, блокировки и deadlock, UPDATE/DELETE и VACUUM, connection pool, пагинация, bulk-операции, UPSERT, ограничения и CHECK, нормализация и production-диагностика.

1. Ментальная модель PostgreSQL

Запрос проходит не напрямую «SQL → данные». PostgreSQL сначала разбирает запрос, строит план и только потом выполняет его.

```
SQL
↓
parser
↓
analyzer / rewrite
↓
planner / optimizer
↓
execution plan
↓
executor
↓
heap / indexes / buffers
↓
result
```

Главный объект мышления при оптимизации — не текст SQL сам по себе, а выбранный план выполнения и реальные данные.

2. SELECT: WHERE, ORDER BY, LIMIT

WHERE уменьшает множество строк. ORDER BY требует порядка. LIMIT ограничивает выдачу, но не обязательно означает, что PostgreSQL прочитает только LIMIT строк.

Если есть только индекс по user_id, запрос вида «user_id = 42 ORDER BY created_at DESC LIMIT 20» может найти много строк пользователя, а затем выполнить сортировку или просмотреть большое количество подходящих строк до получения первых 20.

```
-- запрос из практики
SELECT *
FROM orders
WHERE user_id = 42
ORDER BY created_at DESC
LIMIT 20;
```

Для такого паттерна естественный кандидат — составной индекс по (user_id, created_at), потому что сначала ограничивается user_id, а внутри него уже доступен нужный порядок по времени.

```
CREATE INDEX ON orders (user_id, created_at DESC);
```

3. JOIN: базовая модель



INNER JOIN оставляет совпавшие пары. LEFT JOIN сохраняет все строки левой таблицы и добавляет совпадения справа, а при отсутствии совпадения справа появляются NULL.

Для JOIN PostgreSQL может выбирать разные алгоритмы: Nested Loop, Hash Join, Merge Join. Выбор зависит от оценок количества строк, стоимости доступа, сортировки, памяти и доступных индексов.

Nested Loop:
outer row → поиск matching rows в inner

Hash Join:
build hash(inner) → scan outer → hash lookup

Merge Join:
sorted A + sorted B → проход двумя указателями

4. EXISTS и JOIN

EXISTS отвечает на вопрос «существует ли хотя бы одна подходящая строка». Это часто естественнее, когда не нужны данные из второй таблицы и нужно только проверить наличие.

```
SELECT *
FROM users u
WHERE EXISTS (
  SELECT 1
  FROM orders o
  WHERE o.user_id = u.id
);
```

JOIN может размножить строки, если справа несколько совпадений. EXISTS проверяет наличие и не требует возвращать каждое совпадение как отдельную строку.

5. Индексы: зачем и что они меняют

Индекс — дополнительная структура данных, позволяющая находить строки дешевле полного последовательного сканирования при подходящем предикате/порядке.

Без подходящего индекса:
table → Seq Scan → много строк

С индексом:
query condition → Index Scan / Bitmap Scan → нужные heap rows

Индекс ускоряет чтение не бесплатно: он занимает место, должен обновляться при INSERT/UPDATE/DELETE и может ухудшать запись. Поэтому индекс создают под реальные паттерны запросов.

6. B-tree

B-tree — основной универсальный тип индекса PostgreSQL. Он хорошо подходит для сравнений, диапазонов и упорядоченного доступа: =, <, <=, >, >= и ORDER BY в подходящих случаях.

```
root
/ \
... ..
/ \
sorted leaf pages → → →
```

Индекс можно мысленно представить как отсортированный навигатор к страницам таблицы. PostgreSQL быстро находит диапазон ключей, вместо чтения всей таблицы.

7. Составные индексы

Порядок колонок в составном индексе критичен. Для индекса (A, B) структура организована сначала по A, затем по B внутри одинакового A.

```
INDEX (user_id, created_at)
```

```
user 1: created_at ...
user 1: created_at ...
user 2: created_at ...
user 3: created_at ...
```

Поэтому индекс (user_id, created_at) особенно хорош для запросов с равенством/ограничением по user_id и сортировкой/диапазоном по created_at. Это не означает, что тот же индекс одинаково хорош для запроса, фильтрующего только по второй колонке.

Выбор индекса всегда связываем с конкретным workload: WHERE, JOIN, ORDER BY, LIMIT и кардинальностью.

8. EXPLAIN и EXPLAIN ANALYZE

EXPLAIN показывает предполагаемый план. EXPLAIN ANALYZE реально выполняет запрос и добавляет фактические показатели.

```
EXPLAIN
SELECT ...;
```

```
EXPLAIN ANALYZE
SELECT ...;
```

В плане особенно полезно смотреть: estimated rows vs actual rows, cost, actual time, loops, выбранный Scan/Join и узкие места.

```
estimated rows
↓
planner believes N rows
```

```
actual rows
↓
executor actually got M rows
```

N сильно отличается от M

↓

проверяем статистику / распределение данных / селективность

Cost — внутренняя оценка стоимости, а не миллисекунды. actual time — фактическое время выполнения узла.

9. Планировщик и статистика

Planner выбирает план на основании статистики о данных. Если статистика плохо описывает реальное распределение, оценки cardinality могут быть ошибочными и оптимизатор может выбрать неудачный план.

данные

↓

ANALYZE

↓

статистика

↓

planner

↓

оценка selectivity / rows

↓

выбор плана

В диалоге отдельно разобрался n_distinct. Это характеристика числа различных значений в колонке/оценки количества distinct values. При сильно неравномерном распределении важны также most_common_vals и most_common_freqs.

ANALYZE обновляет статистику; сам ANALYZE не «возвращает таблицу результата» как обычный SELECT — его результатом является обновление статистики для planner.

10. Seq Scan, Index Scan, Bitmap Scan

Seq Scan читает таблицу последовательно. Index Scan идёт через индекс к конкретным строкам. Bitmap Scan сначала собирает bitmap подходящих heap-страниц/строк, а затем читает таблицу более группированно.

Seq Scan:

table pages → → → →

Index Scan:

index → row → heap

Bitmap:

index → bitmap of matching pages

↓

heap pages

Индекс не гарантирует использование. Если условие возвращает большую долю таблицы, Seq Scan может оказаться дешевле.

11. Транзакция

Транзакция объединяет несколько операций в одну логическую единицу. Главная идея ACID: атомарность, согласованность, изоляция, долговечность.

```
BEGIN
UPDATE ...
INSERT ...
COMMIT
```

или

```
BEGIN
...
ROLLBACK
```

Если транзакция откатывается, её изменения не становятся частью подтверждённого состояния БД. COMMIT фиксирует результат.

12. Изоляция транзакций

Уровень изоляции определяет, какие эффекты конкурентных транзакций допустимо наблюдать.

READ COMMITTED

↓

каждый statement видит подходящий snapshot

REPEATABLE READ

↓

транзакция работает с согласованным snapshot

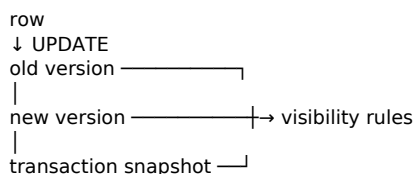
SERIALIZABLE

↓
БД обеспечивает эквивалентность
сериализованному выполнению,
при конфликте возможен rollback/retry

В PostgreSQL READ COMMITTED — стандартный уровень. Важная деталь: snapshot в READ COMMITTED относится к отдельному statement, поэтому два SELECT в одной транзакции могут увидеть разные committed состояния, если между ними закоммитилась другая транзакция.

13. MVCC

PostgreSQL использует MVCC — Multi-Version Concurrency Control. При изменении строки PostgreSQL не просто «перезаписывает единственную видимую версию»: появляются версии, а snapshot определяет, какая версия видима конкретной транзакции.

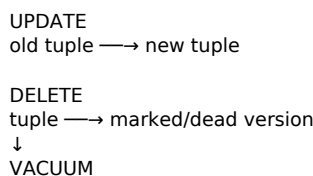


Это позволяет читателям и писателям в типичных случаях не блокировать друг друга как единственный механизм согласованности.

Старые версии не исчезают мгновенно. Их физическая очистка связана с VACUUM/autovacuum.

14. UPDATE и DELETE в контексте MVCC

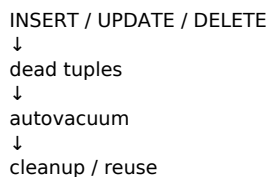
UPDATE создаёт новую версию строки; DELETE делает текущую версию невидимой для новых подходящих snapshot'ов, а старые версии должны быть позже очищены, когда это безопасно.



Поэтому большое число UPDATE/DELETE может порождать dead tuples и создавать дополнительную работу для VACUUM и хранения.

15. VACUUM и autovacuum

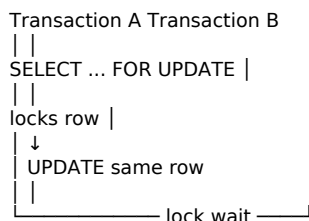
VACUUM освобождает место внутри таблиц для повторного использования и поддерживает здоровье MVCC-структур; autovacuum автоматически запускает обслуживание на основании настроек и активности.



VACUUM — не то же самое, что VACUUM FULL. VACUUM обычно не требует эксклюзивной блокировки всей таблицы и ориентирован на обычное обслуживание; VACUUM FULL перестраивает таблицу и требует значительно более сильной блокировки.

16. Блокировки

Блокировки нужны там, где конкурентные операции должны быть согласованы. В PostgreSQL есть разные типы блокировок, включая row-level locks и table-level locks.



SELECT ... FOR UPDATE блокирует выбранные строки для конфликтующих операций до конца транзакции. В SQLAlchemy это выражается через with_for_update() / соответствующий механизм ORM.

17. Deadlock

Deadlock — цикл ожиданий: А ждёт ресурс В, а В ждёт ресурс А. PostgreSQL обнаруживает deadlock и прерывает одну из транзакций.

```
Transaction A Transaction B
lock row 1 lock row 2
↓ ↓
wait row 2 ←————— wait row 1
DEADLOCK
```

Типичный способ снизить риск — одинаковый порядок захвата ресурсов во всех транзакциях и короткие транзакции. Код должен уметь корректно обрабатывать retry после транзакционных ошибок, когда это применимо.

18. UPSERT / ON CONFLICT

UPSERT позволяет атомарно обработать ситуацию конфликта уникальности: вставить строку или выполнить действие при конфликте.

```
INSERT INTO offers (bank_id, currency, rate)
VALUES (1, 'USD', 89.70)
ON CONFLICT (bank_id, currency)
DO UPDATE SET rate = EXCLUDED.rate;
```

ON CONFLICT использует уникальное ограничение/индекс как механизм обнаружения конфликта. Это предпочтительнее ручного «SELECT → если нет INSERT иначе UPDATE» при конкурентных операциях, если бизнес-логика допускает такой паттерн.

19. Constraints и CHECK

Ограничения позволяют переносить инварианты в БД: PRIMARY KEY, UNIQUE, FOREIGN KEY, NOT NULL, CHECK.

```
CREATE TABLE offers (
  id bigint PRIMARY KEY,
  rate numeric NOT NULL,
  currency text NOT NULL,
  CHECK (rate >= 0)
);
```

CHECK полезен для локальных условий над значениями строки. Ограничения — дополнительный уровень защиты от некорректных данных независимо от конкретного приложения.

20. Нормализация

Нормализация уменьшает дублирование и аномалии вставки/обновления/удаления. При этом в production допускается денормализация, если она осознанно улучшает workload и компенсируется контролем согласованности.

До:
orders(user_id, user_name, ...)

После:
users(id, user_name)
orders(id, user_id, ...)

В одной из обсуждавшихся backend-задач legacy certificate service имел недостаточно нормализованную БД; решение включало нормализацию и миграцию при сохранении бизнес-логики.

21. Pagination: OFFSET vs keyset

OFFSET/LIMIT просты, но при больших OFFSET PostgreSQL должен пройти/пропустить большое количество строк. Keyset pagination использует последнее значение как границу следующей страницы и хорошо работает при наличии подходящего индекса.

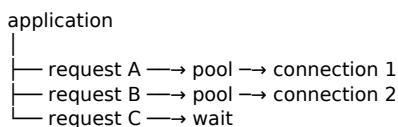
OFFSET:
page 1 → OFFSET 0
page 10000 → пропустить много строк

Keyset:
WHERE created_at < :last_created_at
ORDER BY created_at DESC
LIMIT 20

Для стабильной пагинации при одинаковых timestamps часто нужен дополнительный уникальный tie-breaker, например id.

22. Connection pool

Создание PostgreSQL-соединения дорого по сравнению с использованием уже открытого соединения. Connection pool держит ограниченное число соединений и выдаёт их обработчикам по мере необходимости.



после работы connection возвращается в pool

Слишком большой pool не означает больше производительности: PostgreSQL сам ограничен CPU, I/O, locks и допустимым числом connections.

23. Bulk operations

Массовые операции обычно эффективнее множества отдельных round-trip запросов. В зависимости от задачи используются batch INSERT, COPY, bulk insert/update и другие механизмы.

Важно учитывать размер транзакции, WAL, locks, memory и влияние на конкурирующий workload. «Одна гигантская транзакция» не всегда лучше — её стоимость и последствия нужно оценивать.

24. Production-диагностика медленного запроса

1. Воспроизвести / зафиксировать запрос
- ↓
2. EXPLAIN ANALYZE
- ↓
3. Сравнить estimated rows и actual rows
- ↓
4. Найти дорогой узел
- ↓
5. Проверить индексы / JOIN / сортировку
- ↓
6. Проверить статистику и ANALYZE
- ↓
7. Проверить locks / I/O / connections
- ↓
8. Изменить и повторно измерить

Не начинать с добавления индекса «на всякий случай». Сначала понять workload и план.

25. Очень важный пример: 100M orders

Разбирался запрос: таблица orders примерно на 100 млн строк, фильтр user_id = 42, сортировка created_at DESC, LIMIT 20, а существующий индекс только по user_id.

```
SELECT *
FROM orders
WHERE user_id = 42
ORDER BY created_at DESC
LIMIT 20;
```

```
INDEX (user_id)
↓
найти все строки user 42
↓
если их много:
сортировка / чтение большого набора
↓
LIMIT 20
```

Естественный кандидат для этого access pattern — (user_id, created_at DESC), потому что он одновременно поддерживает фильтрацию по user_id и нужный порядок внутри группы.

26. Что означает «индекс не используется»

Это не обязательно ошибка. Planner сравнивает стоимость вариантов. Если условие малоселективно и нужно прочитать значительную часть таблицы, последовательное чтение может быть дешевле индекса.

selectivity высокая
→ мало строк
→ index часто выгоден

selectivity низкая
→ много строк
→ Seq Scan может быть выгоднее

27. JOIN-алгоритмы: что отвечать кратко

Nested Loop — для каждой строки внешнего набора ищет совпадения во внутреннем; особенно полезен, когда внешний набор небольшой и внутренний поиск дешёвый, например по индексу.

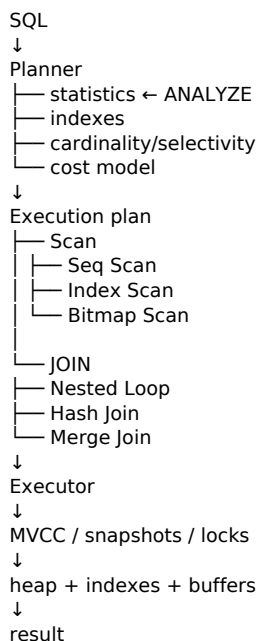
Hash Join — строит hash structure по одной стороне и затем ищет соответствия по другой; часто полезен для equality join.

Merge Join — проходит два отсортированных набора; особенно хорош, когда входы уже отсортированы или сортировка оправдана.

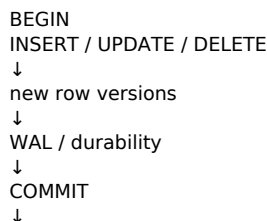
28. Набор коротких формулировок для интервью

Вопрос	Ответ
Что такое MVCC?	Механизм многоверсионности: транзакции работают с версиями строк и snapshot/visibility rules, что позволяет организовать конкурентный доступ без блокировки каждого чтения писателями.
Что делает ANALYZE?	Собирает статистику о данных, которую planner использует для оценки селективности и количества строк и выбора плана.
EXPLAIN vs EXPLAIN ANALYZE?	EXPLAIN показывает план и оценки; EXPLAIN ANALYZE выполняет запрос и показывает фактические показатели.
Зачем составной индекс?	Чтобы оптимизировать конкретный access pattern сразу по нескольким колонкам, учитывая порядок колонок и сочетание WHERE/JOIN/ORDER BY/LIMIT.
Почему индекс может не использоваться?	Planner считает другой план дешевле, например Seq Scan при низкой селективности.
Что такое deadlock?	Циклическое ожидание ресурсов между транзакциями; PostgreSQL обнаруживает его и прерывает одну транзакцию.
Зачем VACUUM?	Для обслуживания MVCC, очистки dead tuples и повторного использования пространства; autovacuum делает это автоматически.
Зачем SELECT FOR UPDATE?	Чтобы заблокировать выбранные строки для конфликтующих изменений до конца транзакции и синхронизировать конкурентную бизнес-операцию.
Почему OFFSET плох на больших страницах?	Нужно пройти/пропустить большое количество строк. Keyset pagination позволяет продолжать от последнего ключа.
Зачем connection pool?	Переиспользовать ограниченный набор соединений и не создавать новое соединение на каждый запрос.

29. Финальная схема: как связывать темы



А изменения данных идут через транзакции:



old versions eventually cleaned by VACUUM

Главная связка для собеседования: плохой запрос → смотрю EXPLAIN ANALYZE → проверяю оценки rows → проверяю статистику → смотрю Scan/Join → проверяю индексы → проверяю locks/I/O/connections → измеряю изменение.